

Računarska vežba 2

Python operatori, tipovi podataka i stringovi

1. Python operatori i tipovi podataka

U računarskoj tehnici, celi i decimalni brojevi se različito zapisuju. Za decimalne brojeve koristi se floating point zapis. Ovo znači da postoji ograničenje u preciznosti zapisa decimalnih brojeva. Tipično je preciznost u Python-u 15 decimala. To znači da, na primer, $7/3=2.33333\dots$, ne može biti reprezentovano sa beskonačnom preciznošću i ako u komandnoj liniji ukucamo $7/3$, dobijamo 2.3333333333333335 , i na petnaestoj decimali imamo grešku zaokruživanja.

Osnovni Python operatori su:

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

Celobrojno deljenje: Obično deljenje $/$, daje očekivani rezultat, npr. $9/4$ daje 2.25

Celobrojno deljenje $//$ odbacuje decimalni deo, pa $9//4$ daje 2

Ostatak pri deljenju (modulo) broja a sa b se zapisuje u obliku $a\%b$. Ako je $a\%b$ jednako 0 , tada je a deljivo sa b , i npr. $9\%2$ jednako je 1 , jer kada 9 podelimo sa 4 dobijamo celobrojno 2, i ostatak 1. Ako je broj a paran, važiće da je mu je ostatak pri deljenju sa 2, odnosno, $a\%2$ jednak 0.

Primer upotrebe - kada želimo da u petlji neku operaciju obavljamo svaki drugi put (uz korišćenje IF, ispitujemo da li je ostatak pri deljenju brojačke promenljive 0, i ako jeste operaciju obavljamo, a ako nije preskačemo).

Moduli – Python biblioteke: Osnovni elementi Python jezika podrazumevaju ključne elemente kao što su `for`, `if`, elementarni operatori, funkcije za unos i prikaz rezultata (`input`, `print`).

Ostali elementi dostupni su u tzv. modulima i moraju se uključiti, korišćenjem `import`, u sledećem obliku:

```
from naziv_modula import sta_ukljucujemo
```

Slučajni celi brojevi – `randint`

Funkcija `randint(a,b)` koja generiše slučajne cele brojeve između a i b (uključujući a i b), dostupna je u modulu `random`, i može se uključiti na sledeći način: `from random import randint`

Primjer upotrebe:

```
from random import randint
x = randint(10, 90)
print('Slučajan broj između 10 i 90: ', x)
```

Svaki ponovni poziv `random` generiše novi slučajni broj između 10 i 90.

Ugrađene funkcije: Funkcije `abs` i `round` su ugrađene, i nije neophodno vršiti njihov import. Funkcija `abs` računa apsolutnu vrednost broja. Na primer, `abs(-11.7)` kao rezultat daje `11.7`

Funkcija `round` vrši zaokruživanje na najbliži ceo broj, ili na najbližu decimalu. Zadaje se u obliku `round(a,n)` gde je prvi argument broj koji se zaokružuje, a drugi određuje na šta se zaokružuje prvi broj. Ako je $n > 0$, zaokruživanje se vrši n pozicija desno od decimalne tačke, za $n < 0$ zaokruživanje se vrši n pozicija levo od decimalne tačke, a za $n = 0$ zaokruživanje je na najbliži ceo broj.

`round(5.8)` kao rezultat daje `6`.

`round(5.2)` kao rezultat daje `5`.

`round(128.5, -2)` kao rezultat daje `100.0`

Modul `math`:

Osnovne matematičke funkcije dostupne su u okviru modula `math`. Njihovo učitavanje se može obaviti naredbom `import math`. Listu učitanih funkcija možemo dobiti komandom `dir(math)`

```
>>> import math
>>> dir(math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin',
 'asinh', 'atan', 'atan2', 'atanh', 'cbrt', 'ceil', 'comb', 'copysign', 'cos', 'cosh',
 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'exp2', 'expm1', 'fabs', 'factorial', 'flo
 or', 'fma', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinit
 e', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2'
 , 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin'
 , 'sinh', 'sqrt', 'sumprod', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
>>>
```

Sadržaj modula `math`:

- Pojedini nazivi fajlova počinju sa `__` (dve donje crte)!!!!
- Obratiti pažnju na Ojlerov broj `e` i konstantu `pi`, odnosno broj π
- Trigonometrijske funkcije su `sin`, `cos`, `tan` (argumenti su u radijanima)
- Inverzne trigonometrijske funkcije su `asin`, `acos`, `atan`
- Hiperboličke funkcije su `sinh`, `cosh`, `tanh`
- Eksponencijalna funkcija `exp`
- Logaritamske funkcije `log`, `log2`, `log10`
- Kvadratni koren `sqrt`
- Stepen `pow(x,y,z=None)`, treći argument je opcion `x**y/z`
- Faktorijel `factorial`
- Zaokruživanje `ceil`, `floor`
- Specijalne konstante `inf`, `nan`
- Najveći zajednički delilac `gcd`
- Funkcija za konverziju radijana u stepene `degrees`
- Funkcija greške `erf` i inverzna funkcija greške `erfc`

Obratiti pažnju da se učitavanje modula može obaviti na više načina:

```
from math import sin, pi, exp #ucitavanje funkcija sin, pi, exp
import math * #ucitavanje svih modula
```

Primer: `math.sin(math.pi/2)`

ZADATAK 1: Izračunati vrednost sume upotrebom `for` komande:

$$s = \sum_{n=1}^{10000} \frac{1}{n^3} = \frac{1}{1^3} + \frac{1}{2^3} + \frac{1}{3^3} + \dots + \frac{1}{10000^3}$$

```
>>> s=0 #inicijalizujemo promenljivu s
>>> for n in range(1,10001): # za n od 1 do 1000
...     s = s + 1/n**3
...
...
>>> s
1.202056898160098
>>>
```

Funkcije `input`, `eval`, `print`:

Osnovna struktura funkcije `input` je:

```
<promjenljiva> = input(<poruka za korisnika>)
```

Funkcija `input` služi za preuzimanje stringova.

```
ime = input('Unesi svoje ime: ')
print('Zdravo, ', ime)
```

U cilju preuzimanja numeričkih vrednosti, koristi se funkcija `eval` koja vrši konverziju stringa u broj. Ako se npr. unese matematički izraz oblika `27`, funkcija `eval` će ga izračunati i pridružiti promenljivoj sa leve strane.

```
N = eval(input('Uneti neki broj: '))
print('Broj na stepen 7 je: ', N^7)
```

Funkcijom `print` vrši se ispis na ekranu: `print('Programski jezik Python')` i kod nje je neophodna upotreba običnih zagrada. Funkcija `print` nakon štampanja rezultata automatski prelazi u novi red. Funkcija `print` može imati više argumenata koji se razdvajaju se zarezom, a pri štampanju, funkcija automatski dodaje razmake između njih.

ZADATAK 2: Napisati program koji traži od korisnika da unese dva broja `x` i `y` a zatim izračunava izraz

$$\ln\left(\frac{|x-y|}{\sqrt{(x+y)^3}}\right)$$

```
>>> x,y = eval(input('Unesite x, y: '))
Unesite x, y: 3,5
>>> izraz = log(abs(x-y)/sqrt((x+y)**3))
>>> print(izraz)
-2.426015131959809
```

Ovde obratiti pažnju na način kako se unose promenljive (metode `input` i `eval`) !!!!!

U prvoj liniji prethodnog programa definisano je šta će program ispisati, odnosno, tražiti od korisnika. Funkcija `input` ispisuje tekst koji je naveden unutar zagrada pod navodnicima, kojim ona zapravo obaveštava korisnika da nešto treba da unese, a zatim preuzima to što je korisnik uneo. Funkcija `eval` omogućava da se ono što je korisnik uneo protumači kao numerička vrednost. Obratiti pažnju da znak `=` označava operator pridruživanja sadržaja sa desne strane u promenljivu `izraz` koja je sa leve strane ovog operatora.

Python je case sensitive, i razlikuje mala i velika slova pa npr. **Masa**, **masa** i **MASA** predstavljaju različite promenljive!

Paziti na uvlačenje (indent) i grupisanje komandi, naročito kod uslovnih izraza:

```
>>> masa = eval(input('Unesite masu u kg: '))
Unesite masu u kg: 45
>>>     print('U funtama to je:', 2.2046*masa)
...
SyntaxError: unexpected indent
>>>
```

ZADATAK 3: Odrediti sadržaj promenljivih **x**, **y** i **z** nakon izvršavanja koda (analizirati kod):

```
x=3
y=4
z=x+y
z=z+1
x=y
y=5
```

```
>>> x=3
>>> y=4
>>> z=x+y
>>> z=z+1
>>> x
3
>>> y
4
>>> z
8
>>> x=y
>>> x
4
```

Sadržaj promenljivih se dobija ukoliko se kod protumači liniju po liniju:

- U promenljivu **x** se smešta vrednost **3**
- U promenljivu **y** se smešta vrednost **4**
- U promenljivu **z** se smešta vrednost **x+y**, odnosno vrednost **7**
- U promenljivu **z** se smešta stara vrednost iz **z** uvećana za 1, odnosno 8
- U promenljivu **x** se smešta sadržaj promenljive **y** odnosno 4
- Promenljiva **y** dobija vrednost 5
- Da li bi bilo dozvoljeno koristiti **x=y=5**?

ZADATAK 4: Napisati kratak program za osnovne aritmetičke operacije upotrebom promenljivih

```
>>> # Deklarisanje promenljivih
>>> num_one = 3
>>> num_two = 4
>>>
>>> #Aritmeticki operatori
>>> total = num_one + num_two
>>> diff = num_two - num_one
>>> product = num_one * num_two
>>> div = num_two / num_one
>>> remainder = num_two % num_one
>>>
>>> # Print vrednosti
>>> print('total: ', total)
total: 7
>>> print('difference: ', diff)
difference: 1
>>> print('product: ', product)
product: 12
>>> print('division: ', div)
division: 1.3333333333333333
>>> print('remainder: ', remainder)
remainder: 1
>>>
```

Osnovni Python operatori poredenja su:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Primeri:

```
print(3 > 2)      # True, jer je 3 vece od 2
print(3 >= 2)     # True, jer je 3 vece od 2
print(3 < 2)      # False, jer je 3 vece od 2
print(2 < 3)      # True, jer je 2 manje od 3
print(2 <= 3)     # True, jer je 2 manje od 3
print(3 == 2)     # False, jer 3 nije jednako 2
print(3 != 2)     # True, jer 3 nije jednako 2
print(len('mango') == len('avocado')) # False
print(len('mango') != len('avocado')) # True
print(len('mango') < len('avocado'))  # True
print(len('milk') != len('meat'))      # False
print(len('milk') == len('meat'))      # True
print(len('tomato') == len('potato'))  # True
print(len('python') > len('dragon'))   # False
```

Funkcija `len()` vraća dužinu stringa.

Python koristi i sledeće operatore poređenja:

- **is**: Vraća True ako su obe promenljive isti objekti (x je y)
- **is not**: Vraća True ako su promenljive nisu isti objekti (x nije y)
- **in**: Vraća True ako lista sadrži određeni element (x je u y)
- **not in**: Vraća True ako lista ne sadrži određeni element

Primeri:

```
print('1 is 1', 1 is 1)          # True - vrednosti podataka su site
print('1 is not 2', 1 is not 2)  # True - jer 1 nije 2
print('A in Petar', 'P' in 'Petar') # True - Karakter P pronađen u stringu
print('S in Petar', 'S' in 'Petar') # False - Nema velikog slova S u stringu
print('coding' in 'coding for all') # True - String sadrži reč
print('a in an:', 'a' in 'an')    # True
print('4 is 2 ** 2:', 4 is 2 ** 2) # True
```

Osnovni Python **logički operatori** su:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

Primeri:

```
print(3 > 2 and 4 > 3) # True - jer su oba iskaza tačna
print(3 > 2 and 4 < 3) # False - jer je drugi iskaz netačan
print(3 < 2 and 4 < 3) # False - jer su oba iskaza netačna
print(3 > 2 or 4 > 3)  # True - jer su oba iskaza tačna
```

```
print(3 > 2 or 4 < 3) # True - jer je jedan od iskaza tačan
print(3 < 2 or 4 < 3) # False - jer su oba iskaza netačna
print(not 3 > 2)      # False - jer je 3 > 2 tačno, i tada not True daje False
print(not True)       # False - Negacija, not operator menja true u false
print(not False)      # True
print(not not True)   # True
print(not not False)  # False
```

ZADACI ZA VEŽBU

- 5) Deklarisati promenljivu koja čuva kompleksan broj.
- 6) Napisati script koji od korisnika zahteva da unese stranicu a , stranicu b i stranicu c trougla, a zatim izračunati njegov obim.

```
perimeter_triangle = int(int(input("Unesi stranicu a: ")) + int(input("Unesi stranicu b: ")) + int(input("Unesi stranicu c: ")))
```

```
print(.....)    Završiti kod!!!
```

- 7) Napisati script koji kao izlaz prikazuje sledeću tabelu (koristiti **for** petlju):

```
1 1 1 1 1
2 1 2 4 8
3 1 3 9 27
4 1 4 16 64
5 1 5 25 125
```

```
for i in range(1, 6):
    print(i, 1, i, i**2, i**3)
```

- 8) Napisati script koji od korisnika traži da unese svoj broj godina. Izračunati broj sekundi koje je proživio. Pretpostaviti da čovek može živeti do 100 godina!!!
- 9) Napisati script koji generiše slučajni decimalni broj između 1 i 10, sa dve decimale, kao npr. 1.23, 3.45, 9.80, i 5.00

```
from random import randint
a = randint(100,1000)
print(a/100)
```

- 10) Jedan način da se pronađu dve poslednje cifre nekog broja je da se odredi ostatak pri deljenju tog broja sa 100. Napisati script koji traži od korisnika da unese eksponent (stepen), a onda pronalazi poslednje dve cifre broja koji se dobija kada se 2 podigne na taj stepen.

```
from math import *
a = eval(input('Uneti stepen broja: '))
broj = 2^a
print('Dve poslednje cifre broja su: ', broj%100)
```

- 11) Napisati program kojim se crta „modularni pravougaonik” kao što je prikazano ispod. Korisnik treba da unese širinu i visinu pravougaonika, a brojevi unutar pravougaonika počinju od 0,

povećavaju se sa leva na desno, ali po modulu 10. Donji primeri pokazuju pravougaonike dimenzija 3×5 i 4×8 . Preporuka je koristiti dvostruko ugnežšenu **for** petlju!

```
0 1 2 3 4
5 6 7 8 9
0 1 2 3 4
```

```
0 1 2 3 4 5 6 7
8 9 0 1 2 3 4 5
6 7 8 9 0 1 2 3
4 5 6 7 8 9 0 1
```

```
a = eval(input("Uneti visinu pravougaonika: "))
b = eval(input("Uneti sirinu pravougaonika: "))
br = 0
for i in range(a):
    for j in range(b):
        print(br%10,end=" ")
        br = br + 1
    print()
```

2. Python – rad sa stringovima

Stringovi su nizovi karaktera i predstavljaju tip podatka. Za definisanje stringa, mogu se koristiti jednostruki navodnici (`'`), dvostruki navodnici (`"`) kao i trostruki navodnici (`"""`) koji se koriste u slučaju kada se string proteže u više linija.

```
s1='SPA'
```

```
s2="Hello Python studenti"
```

```
s3="""Život je neshvatljivo čudo, stalno se troši i osipa,
a ipak traje i stoji čvrsto - kao na Drini ćuprija"""
```

Za unošenje stringa sa tastature, koristi se `input` funkcija, ali bez dodatne upotrebe `eval` funkcije:

```
s=input('Unesite neki tekst: ')
```

Dužina stringa se dobija upotrebom funkcije `len` tako da npr.

```
len('Novak Djokovic')
```

kao rezultat daje 14.

Nadovezivanje stringova obavlja korišćenjem operatora `+` pa će npr. biti da `'jedan'+ '2'+ 'tri'` kao rezultat daje `'jedan2tri'`

Ponavljjanje stringova se obavlja operatorom `*` tako da npr. `'#'*5` formira string `'#####'`

ZADATAK 1: Napisati program koji od korisnika traži unos 10 slova, i koji formira string sastavljen samo od unetih samoglasnika (zadatak uraditi pomoću **for** i **if** iskaza)

```
>>> s=''
>>> for i in range(10):
...     t=input('Unesite slovo:')
...     if t=='a' or t=='e' or t=='i' or t=='o' or t=='u':
...         s=s+t
...     print(s)
```

ZADATAK 2: Napisati program koji od korisnika traži unos 10 karaktera, i koji u string *s* smešta unete cifre. (zadatak uraditi pomoću **for** i **if** iskaza)

Zadatak uraditi slično prethodnom, uz moguće rešenje uslovnog iskaza IF

```
... if t in '0123456789':
```

Indeksiranje stringa:

Elementima stringa se pristupa korišćenjem zagrada `[]`. Indeksiranje počinje od 0. Moguće su opcije sa pozitivnim i negativnim indeksima. U reči 'Python', `s[0]` ima vrednost 'P', dok `s[5]` ima vrednost 'n', odnosno, `s[-1]` ima vrednost 'n', dok `s[-6]` ima vrednost 'P',

-6	-5	-4	-3	-2	-1
['P',	'y',	't',	'h',	'o',	'n']
0	1	2	3	4	5

Osnovna struktura je oblika `[početna pozicija: krajnja pozicija + 1: korak]`

Bilo koja od pozicija može biti izostavljena. Ako se izostavi početna pozicija, podrazumevana vrednost je 0 (ide se od početka stringa). Ako se izostavi krajnja pozicija - podrazumeva se kraj stringa. Negativni indeksi odnose se na kraj stringa (udaljenost od kraja). Treći, opcioni argument je korak.

Posmatrajmo string 'Matematika'.

indeks: 0 1 2 3 4 5 6 7 8 9

karakter: M a t e m a t i k a

Kod	Rezultat	Objašnjenje
<code>s[2:5]</code>	'tem'	Slova čiji je indeks 2,3,4
<code>s[:5]</code>	'Matem'	Prvih 5 slova (indeks 0 do 4)
<code>s[5:]</code>	'atika'	Slova od indeksa 5 do kraja
<code>s[-2:]</code>	'ka'	Poslednja dva slova
<code>s[:]</code>	'Matematika'	Celi string
<code>s[1:7:2]</code>	'aea'	Slova od 1 do 6 sa korakom 2.
<code>s[::-1]</code>	'akitametaM'	Obrnut zapis stringa.

String metode:

String metoda	Opis
<code>lower()</code>	Vraća string sa svim slovima konvertovanim u mala slova
<code>islower()</code>	Proverava da li su svi karakteri stringa malo slovo
<code>upper()</code>	Vraća string sa svim slovima konvertovanim u velika slova
<code>isupper()</code>	Proverava da li su svi karakteri stringa veliko slovo
<code>replace(x,y)</code>	Vraća string kod kojeg su zamenjena sva pojavljivanja <i>x</i> sa <i>y</i>
<code>count(x)</code>	Broji koliko se puta pojavljuje <i>x</i> u stringu
<code>index(x)</code>	Kao rezultat vraća indeks pozicije prvog pojavljivanja <i>x</i> u stringu

Strukture podataka i algoritmi

<code>rindex(x)</code>	Kao rezultat vraća indeks pozicije poslednjeg pojavljivanja <i>x</i> u stringu
<code>isalpha()</code>	Vraća True ako je svaki karakter u stringu slovo (malo ili veliko)
<code>isdigit()</code>	Vraća True ako je svaki karakter u stringu cifra
<code>isdecimal()</code>	Proverava da li su svi karakteri stringa decimalni brojevi (0-9)
<code>isnumeric()</code>	Proverava da li su svi karakteri stringa brojevi ili relacija sa brojevima
<code>isalnum()</code>	Vraća True ako je svaki karakter u stringu cifra ili slovo
<code>capitalize()</code>	Konvertuje prvi karakter stringa u veliko slovo
<code>startswith()</code>	Proverava da li string počinje specifičnim karakterima
<code>endswith()</code>	Proverava da li se string završava specifičnim karakterima
<code>expandtabs()</code>	Zamenjuje tab karakter sa space, default tab je 8
<code>find()</code>	Vraća indeks prvog pronađenog karaktera, ako ne vraća -1
<code>rfind()</code>	Vraća indeks poslednjeg pronađenog karaktera, ako ne vraća -1
<code>format()</code>	Formatiranje stringa za bolji prikaz
<code>isidentifier()</code>	Proverava da li je ime promenljive ispravno
<code>join()</code>	Vraća povezane stringove
<code>strip()</code>	Uklanja sve navedene karaktere iz stringa od početka do kraja
<code>replace()</code>	Zamenjuje karaktere stringa navedenim
<code>split()</code>	Deljenje stringa razmakom ili nekim drugim znakom
<code>title()</code>	Vraća "title cased" string
<code>swapcase()</code>	Konvertuje sve uppercase karaktere stringa u lowercase i obrnuto

Primeri:

```
challenge = 'thirty days of python'
print(challenge.count('y')) # 3
print(challenge.count('y', 7, 14)) # 1
print(challenge.count('th')) # 2
```

```
challenge = 'thirty days of python'
print(challenge.capitalize()) # 'Thirty days of python'
```

```
challenge = 'thirty days of python'
print(challenge.endswith('on')) # True
print(challenge.endswith('tion')) # False
```

```
challenge = 'thirty\tdays\totf\tpython'
print(challenge.expandtabs()) # 'thirty  days    of      python'
print(challenge.expandtabs(10)) # 'thirty    days      of          python'
```

```
challenge = 'thirty days of python'
print(challenge.find('y')) # 5
print(challenge.find('th')) # 0
```

```
challenge = 'thirty days of python'
print(challenge.rfind('y')) # 16
print(challenge.rfind('th')) # 17
```

```
challenge = 'thirty days of python'
sub_string = 'da'
print(challenge.index(sub_string)) # 7
print(challenge.index(sub_string, 9)) # error
```

```
challenge = 'thirty days of python'
sub_string = 'da'
print(challenge.rindex(sub_string)) # 7
print(challenge.rindex(sub_string, 9)) # error
print(challenge.rindex('on', 8)) # 19

challenge = '30DaysPython'
print(challenge.isalnum()) # True
challenge = 'thirty days of python 2019'
print(challenge.isalnum()) # False

challenge = 'thirty days of python'
print(challenge.isalpha()) # False, space is once again excluded
challenge = 'ThirtyDaysPython'
print(challenge.isalpha()) # True
num = '123'
print(num.isalpha()) # False

challenge = '30DaysOfPython'
print(challenge.isidentifier()) # False, because it starts with a number
challenge = 'thirty_days_of_python'
print(challenge.isidentifier()) # True

challenge = 'thirty days of python'
print(challenge.isupper()) # False
challenge = 'THIRTY DAYS OF PYTHON'
print(challenge.isupper()) # True

web_tech = ['HTML', 'CSS', 'JavaScript', 'React']
result = '# '.join(web_tech)
print(result) # 'HTML# CSS# JavaScript# React'

challenge = 'thirty days of pythoonnn'
print(challenge.strip('noth')) # 'irty days of py'

challenge = 'thirty days of python'
print(challenge.split()) # ['thirty', 'days', 'of', 'python']
challenge = 'thirty, days, of, python'
print(challenge.split(',')) # ['thirty', 'days', 'of', 'python']

challenge = 'thirty days of python'
print(challenge.startswith('thirty')) # True
challenge = '30 days of python'
print(challenge.startswith('thirty')) # False
```

Escape sekvence stringova:

Escape sekvence se koriste za formatiranje stringova kao u drugim programskim jezicima, navođenjem znaka iza (*back slash*) kose crte \ :

- \n: novi red
- \t: Tab (8 space znakova)
- \\: Back slash
- \': Jednostruki navodnik (')
- \": Dvostruki navodnik (")

```
print('I hope everyone is enjoying the Python Challenge.\nAre you ?') # line break
print('Days\tTopics\tExercises') # adding tab space or 4 spaces
print('Day 1\t5\t5')
print('Day 2\t6\t20')
print('Day 3\t5\t23')
print('Day 4\t1\t35')
print('This is a backslash symbol (\\)') # To write a backslash
print('In every programming language it starts with \"Hello, World!\"') # to write a double
quote inside a single quote
```

output

I hope every one is enjoying the Python Challenge.

Are you ?

Days	Topics	Exercises
------	--------	-----------

Day 1	5	5
-------	---	---

Day 2	6	20
-------	---	----

Day 3	5	23
-------	---	----

Day 4	1	35
-------	---	----

This is a backslash symbol (\\)

In every programming language it starts with "Hello, World!"

Formatiranje stringa (% operator)

"%" operator se koristi za formatiranje promenljivih u formi torke (tuple) i upotrebljavaju se specijalni simboli "%s", "%d", "%f" i "%.number of digitsf"

- %s - String (ili bilo koji string objekat, može biti i broj)
- %d - Celi brojevi
- %f – Realni brojevi
- "%.number of digitsf" – Realni brojevi sa određenom tačnošću

Strings only

```
first_name = 'Petar'
```

```
last_name = 'Petrovic'
```

```
language = 'Python'
```

```
formatted_string = 'I am %s %s. I teach %s' %(first_name, last_name, language)
```

```
print(formatted_string)
```

Strings and numbers

```
radius = 10
```

```
pi = 3.14
```

```
area = pi * radius ** 2
```

```
formatted_string = 'The area of circle with a radius %d is %.2f.' %(radius, area) # 2 refers
the 2 significant digits after the point
```

```
python_libraries = ['Django', 'Flask', 'NumPy', 'Matplotlib','Pandas']
```

```
formatted_string = 'The following are python libraries:%s' % (python_libraries)
```

```
print(formatted_string) # "The following are python libraries:['Django', 'Flask', 'NumPy',
'Matplotlib', 'Pandas']"
```

Formatiranje stringa ili interpolacija (f-strings):

```
a = 4
b = 3
print(f'{a} + {b} = {a + b}')
print(f'{a} - {b} = {a - b}')
print(f'{a} * {b} = {a * b}')
print(f'{a} / {b} = {a / b:.2f}')
print(f'{a} % {b} = {a % b}')
print(f'{a} // {b} = {a // b}')
print(f'{a} ** {b} = {a ** b}')
```

Formatiranje stringa (str.format):

```
a = 4
b = 3

print('{} + {} = {}'.format(a, b, a + b))
print('{} - {} = {}'.format(a, b, a - b))
print('{} * {} = {}'.format(a, b, a * b))
print('{} / {} = {:.2f}'.format(a, b, a / b)) # limits it to two digits after decimal
print('{} % {} = {}'.format(a, b, a % b))
print('{} // {} = {}'.format(a, b, a // b))
print('{} ** {} = {}'.format(a, b, a ** b))

# output
4 + 3 = 7
4 - 3 = 1
4 * 3 = 12
4 / 3 = 1.33
4 % 3 = 1
4 // 3 = 1
4 ** 3 = 64
```

ZADACI ZA VEŽBU

- 1) Napisati program koji od korisnika traži da unese string. Program zatim treba da štampa sledeće:
 - a) Ukupan broj karaktera u stringu
 - b) Sting ponavljen 10 puta
 - c) Prvi karakter u stringu
 - d) Prva tri karaktera iz stringa
 - e) Poslednja tri karaktera iz stringa
 - f) String sa karakterima u obrnutom redosledu (palindrom)
 - g) Sedmi karakter u stringu ako string ima dovoljnu dužinu, a u suprotnom štampati poruku o grešci
 - h) String kod koga su izostavljeni prvi i poslednji karakter
 - i) String sa svim velikim slovima
 - j) String u kojem je svako slovo a zamenjeno sa e
 - k) String u kojem je svako slovo zamenjeno razmakom

2) Pomoću **f-strings** formata napisati script za ispis osnovnih aritmetičkih operacija brojeva 8 i 6:

```
>>> print(f"8 + 6 = {8 + 6}\n"
...       f"8 - 6 = {8 - 6}\n"
...       f"8 * 6 = {8 * 6}\n"
...       f"8 / 6 = {8 / 6:.2f}\n"
...       f"8 % 6 = {8 % 6}\n"
...       f"8 // 6 = {8 // 6}\n"
...       f"8 ** 6 = {8 ** 6}\n")
8 + 6 = 14
8 - 6 = 2
8 * 6 = 48
8 / 6 = 1.33
8 % 6 = 2
8 // 6 = 1
8 ** 6 = 262144
>>>
```

3) Koristeći tab escape sekvence formirati sledeću formu.

Ime	Godine	Zemlja	Grad
Petar	45	Srbija	Novi Sad

```
>>> print(f"Ime\tGodine\tZemlja\tGrad\n"
...       f"Petar\t45\tSrbija\tNovi Sad")
Ime      Godine  Zemlja  Grad
Petar    45      Srbija  Novi Sad
>>>
```

4) Navedena lista sadrži imena pojedinih Python biblioteka: ['Django', 'Flask', 'Bottle', 'Pyramid', 'Falcon']. Metodom **join** formirati niz elemenata liste razdvojenih prazninom (space).

```
>>> lib_list = ['Django', 'Flask', 'Bottle', 'Pyramid', 'Falcon']
>>> print(" ".join(lib_list))
Django Flask Bottle Pyramid Falcon
>>>
```

5) Napisati program koji traži od korisnika da unese string, a onda štampa sve pozicije na kojima se nalazi slovo 'a' u stringu. String na kraju treba i da štampa broj pojavljivanja slova 'a'.

```
brojac=0
s = input('Unesite string: ')
for i in range(len(s)):
    if s[i]=='a':
        print(i)
        brojac=brojac+1
print('Imamo ukupno',brojac, 'slova \'a\'')
```

Kroz string se prolazi slovo po slovo. Promenljiva **i** prati indeks karaktera u stringu. U četvrtoj liniji koda se proverava da li je tekući karakter na poziciji **i** zapravo slovo 'a' i ako jeste, štampa mu se pozicija, dok se brojačka promenljiva inkrementira. Obratiti pažnju upotrebu escape karaktera u naredbi **print** na kraju programa.

6) Napisati program koji od korisnika traži unos stringa, a koji zatim štampa novi string u kojem je ponovljeno svako slovo iz stringa koji je uneo korisnik.

```
.....
duplirani_string=''
for i in range(len(s)):
    duplirani_string=duplirani_string+s[i]*2
.....
```

- 7) Napisati program koji generiše tekst od 26 linija (koliko je slova abecede), čiji je raspored prikazan ispod:

```
abcdefghijklmnopqrstuvxyz  
bcdefghijklmnopqrstuvxyz  
cdefghijklmnopqrstuvxyzab  
.....  
yzabcdefghijklmnopqrstuvwx  
zabcdefghijklmnopqrstuvwxy
```

```
s='abcdefghijklmnopqrstuvxyz'  
for i in range(len(s)):  
    print(s[i:],s[:i],sep='')
```

- 8) Napisati program koji od korisnika traži unos stringa `s` koji sadrži decimalni broj, i koji zatim štampa samo decimalni deo tog broja. Ako korisnik npr. unese '2.7182818' (unos se u vidu stringa!), kao rezultat se štampa: 7182818.

```
s=input('Unesite neki broj: ')  
print(s[s.index('.')+1:])
```